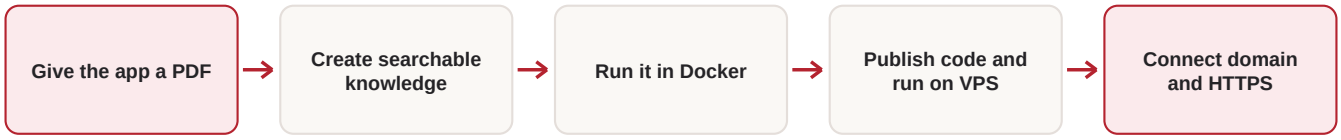
STANDALONE PROJECT GUIDE | BEGINNER EDITION

From a PDF to a live AI assistant

Rebuild the Spotify Architecture Assistant, test it on your computer, and publish it securely at sedeploy.tech.

Your finished project
A document question-answering app at https://sedeploy.tech. This guide explains the project from PDF indexing through the live site and future updates.

THE COMPLETE JOURNEY



How to use this guide

Follow the numbered steps in order the first time. You will learn what each part does, create the same named files, check the app locally, then move it to the server. The final pages give you a repeatable change routine and a prompt for an assistant to coach you one step at a time.

A 45-minute assisted route

Time	Work
0-5 min	Check tools, accounts, PDF, and secret files.
5-15 min	Prepare the PDF and build the Hybrid RAG indexes.
15-23 min	Build the Docker image and test localhost:8501.
23-30 min	Check GitHub code and publish the approved version.
30-39 min	Pull and run the app on the VPS.
39-45 min	Check DNS, Traefik, HTTPS, and ask a test question.

Made for this project: SocialEagleDeploy, Docker Desktop, GitHub, the Hostinger Ubuntu VPS, sedeploy.tech, and the existing Traefik proxy.

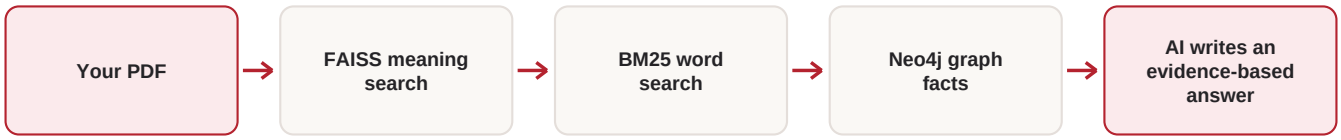
1. One-page map: what are we building?

The app answers questions using information from a Spotify architecture PDF. Each tool has one job. Together they turn the PDF into a searchable assistant and put that assistant on the internet.

The five ideas

Name	Plain-language meaning	In this project
Hybrid RAG	RAG means retrieval-augmented generation: find relevant document evidence first, then ask an AI model to write an answer. Hybrid means it uses more than one kind of search.	FAISS meaning search + BM25 word search + Neo4j graph facts, combined for answers.
Docker	A packaged box containing the app and the software it needs. It runs the same way on a laptop and a server.	One container runs Streamlit and its Python question-answering code on port 8501.
GitHub	An online copy of the project history, so the server can get the version you approved.	The repository is nithyakalaiovian/SocialEagleDeploy on the main branch.
VPS	A rented computer on the internet that stays on.	The Hostinger Ubuntu server at 187.127.134.163 runs Docker.
DNS + HTTPS	DNS points a human-friendly name to the server. HTTPS encrypts browser traffic and confirms the website certificate.	sedeploy.tech points to the VPS; Traefik routes the request and obtains a Let's Encrypt certificate.

How a question travels



How a visitor reaches the app

Remember the two journeys
The first journey prepares knowledge from the PDF. The second journey serves that knowledge to a visitor. Docker packages the app; GitHub carries approved code; the VPS keeps it online.

2. Project folders and how the pieces fit

Use the exact names below. Windows paths use backslashes; the Linux server uses forward slashes.

Important server detail
The app listens on port 8501. Traefik already owns ports 80 and 443 on this VPS and provides domain routing and HTTPS. Do not install a second web proxy on those ports.

The visitor request path

Project layout

```
E:\SocialEagleDeploy\
|-- spotify_web_app_architecture.pdf  Source document for indexing
|-- 01Frontend\app.py            Streamlit interface
|-- 02Backend\rag.py             Question answering and hybrid search
|-- 02Backend\ingest.py          Document ingestion utility
|-- 02Backend\chunks.json        Saved text chunks
|-- 02Backend\faiss_index\      Saved FAISS index files
|-- Dockerfile                      Builds the single app image
|-- compose.yaml                    Runs the app and Traefik route labels
|-- requirements.txt                Python dependencies
|-- .env                            Local secrets; never commit
|-- .gitignore                      Keeps secrets and Python cache out of Git
\-- .dockerignore                  Keeps local-only files out of the image
```

Where each part runs

Part	Location and role
Streamlit UI + RAG backend code	One Docker container on the VPS, listening inside the container on 8501.
Traefik	Existing host-network reverse proxy. Routes the two domain names to the app and handles TLS.
OpenAI and Neo4j	External services configured through environment variables in .env.
GitHub	Stores source code. The server pulls code from the private repository.

3. Essential tools and credentials

Collect access before beginning. This guide lists credential types and variable names only; enter the actual values privately on your own devices.

Tools

Tool or account	Purpose
Visual Studio Code	Edit app.py, Dockerfile, compose.yaml, and documentation.
Docker Desktop	Build and run the app locally before publishing.
PowerShell and Git	Run local Docker and Git commands from E:\SocialEagleDeploy.
GitHub account	Store the source repository: nithyakalaiovian/SocialEagleDeploy.
Hostinger VPS and browser terminal	Run Git and Docker Compose on the Ubuntu server. The server project folder is /opt/social-eagle-git.
Hostinger DNS controls and a browser	Point sedeploy.tech to the server and verify the live HTTPS site.

Credentials needed

Credential	Why it is needed	Keep it in
OPENAI_API_KEY	Generate embeddings and/or answers, according to backend configuration.	Local .env and VPS .env
NEO4J_URI, NEO4J_USERNAME, NEO4J_PASSWORD, NEO4J_DATABASE	Connect the app to the Neo4j knowledge graph.	Local .env and VPS .env
GitHub fine-grained PAT	Clone or pull the private repo from the VPS. Restrict it to this repo and read-only Contents/Metadata if only pulling.	Password manager; enter at Git prompts
Hostinger account / VPS access	Open the VPS browser terminal and manage DNS.	Password manager

Secret-handling rule
Never paste a real API key, database password, GitHub token, or VPS password into this guide, a screenshot, GitHub, or a chat. Keep .env out of Git. On the VPS, protect it with chmod 600 .env.

4. Build Hybrid RAG from the PDF

This is the preparation step: the program reads the PDF once and saves search data. The app uses that saved data each time a visitor asks a question.

What happens to the document

Step	What the computer does	Saved result
1. Read	Opens the text pages of spotify_web_app_architecture.pdf. This works best when you can select and copy text from the PDF.	Page text with page information.
2. Split	Cuts long pages into smaller overlapping pieces so a fact near a page boundary is less likely to be lost. This project uses 400-character pieces with 100-character overlap.	Searchable text pieces.
3. Meaning search	Turns each piece into a numerical meaning pattern using OpenAI embeddings. FAISS compares a question with those patterns.	02Backend/faiss_index/
4. Word search	Keeps the text pieces so BM25 can find important matching words.	02Backend/chunks.json
5. Knowledge graph	Asks an AI model to find named things and their relationships, then saves those facts to Neo4j.	Neo4j graph database.

Do this in VS Code

Put the source PDF in the project root beside Dockerfile and compose.yaml. The default filename is spotify_web_app_architecture.pdf. Open the project folder E:\SocialEagleDeploy in VS Code. Create and activate a Python environment in the VS Code terminal, then install the project requirements:

```
cd E:\SocialEagleDeploy
py -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
python 02Backend/ingest.py spotify_web_app_architecture.pdf
```

Expected result
The terminal reports that it prepared searchable text pieces and saved the FAISS index, BM25 text, and Neo4j graph. Check for 02Backend/chunks.json and the two files in 02Backend/faiss_index/. The local .env must contain working OpenAI and Neo4j settings before ingestion.

If PowerShell blocks activation
Use the current VS Code terminal session only: run Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass, then activate the environment again. This temporary setting ends when that terminal closes.

5. Package the app with Docker

These two root files describe how to build and run the app. Use the copies in the project repository; you do not need to retype them if they already exist.

Root Dockerfile

```
FROM python:3.11-slim
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1
WORKDIR /app
RUN apt-get update \
    && apt-get install -y --no-install-recommends libgomp1 \
    && rm -rf /var/lib/apt/lists/*
COPY requirements.txt .
RUN python -m pip install --no-cache-dir -r requirements.txt
COPY 01Frontend ./01Frontend
COPY 02Backend ./02Backend
EXPOSE 8501
CMD ["python", "-m", "streamlit", "run", "01Frontend/app.py", "--server.address=0.0.0.0", "--server.port=8501"]
```

Root compose.yaml

```
services:
  social-eagle:
    build:
      context: .
      dockerfile: Dockerfile
    ports:
      - "8501:8501"
    env_file:
      - .env
    restart: unless-stopped
    labels:
      - "traefik.enable=true"
      - "traefik.http.routers.social-eagle.rule=Host(`sedeploy.tech`) || Host(`www.sedeploy.tech`)"
      - "traefik.http.routers.social-eagle.entrypoints=websecure"
      - "traefik.http.routers.social-eagle.tls=true"
      - "traefik.http.routers.social-eagle.tls.certresolver=letsencrypt"
      - "traefik.http.services.social-eagle.loadbalancer.server.port=8501"
```

The labels matter
They tell the existing Traefik proxy to route sedeploy.tech and www.sedeploy.tech to this container on port 8501, and to use its existing Let's Encrypt resolver named letsencrypt.

6. Run it in Docker Desktop

Do this on the Windows computer before updating the VPS. Keep Docker Desktop open and wait until its engine is ready.

Create local environment settings first
Create .env in E:\SocialEagleDeploy with the variable names listed on page 4 and your private values. compose.yaml reads this file. Keep it out of Git and never share its contents.

Local test sequence

Step	Action
1	Open PowerShell and enter the project root: E:\SocialEagleDeploy.
2	Validate Compose without printing expanded environment values: docker compose config -q
3	Build and start the container: docker compose up --build -d
4	Check that it is running: docker compose ps
5	Open http://localhost:8501. Test a known document question, the FAQ picker, and status display.
6	If needed, inspect app logs: docker compose logs --tail=50 social-eagle

Local pass condition
The app loads at localhost:8501, the question box accepts typed or selected FAQ questions, and an answer/status is displayed. Fix local errors before proceeding.

Protect secrets before committing

The root .gitignore must include .env, *.env, and an exception for .env.example if you use one. Before staging, run git status --short and make sure .env is not listed. Never use git add . until you have checked the file list.

Initial GitHub setup, if starting from a new copy

Keep this project in its own folder, E:\SocialEagleDeploy, separate from the larger training folder. Check the repository root with git rev-parse --show-toplevel; it should report E:/SocialEagleDeploy. This avoided accidentally adding unrelated training files.

```
cd E:\SocialEagleDeploy
git init
git add .gitignore .dockerignore Dockerfile compose.yaml requirements.txt 01Frontend 02Backend
git status --short   # verify .env is NOT listed
git commit -m "Add Social Eagle Docker app"
git branch -M main
git remote add origin https://github.com/nithyakalaiovian/SocialEagleDeploy.git
git push -u origin main
```

For an existing clone, do not repeat git init or add the remote. Use the normal change-and-push loop on page 8.

7. First deployment on the VPS

These steps reflect the current Hostinger Ubuntu VPS and the installed Docker/Git setup.

Prepare a new VPS

In Hostinger, create an Ubuntu LTS VPS, save its public IP, and open the browser terminal. Update Ubuntu and install Git. Install Docker Engine and the Compose plugin by following Docker's current Ubuntu instructions. Docker changes its installation steps over time, so the official page is the safe source. This project used an Ubuntu VPS with Git and Docker already installed.

```
apt update
apt install -y git
git --version
docker --version
docker compose version
```

Official Docker Engine installation for Ubuntu

Open and check the VPS

Use Hostinger's browser terminal to get a root shell. The version checks above should print version numbers. If the server is new, the existing Traefik proxy described on the next pages must also be installed and configured once before this app can receive HTTPS traffic.

```
git --version
docker --version
docker compose version
```

Get the source code

Open Hostinger browser terminal (this is the method used in this deployment). If the repository is private, GitHub asks for your username and PAT during clone. Enter the PAT at the password prompt; GitHub account passwords do not work for Git over HTTPS. Never put the PAT directly in a command or URL.

```
cd /opt
git clone https://github.com/nithyakalaiovian/SocialEagleDeploy.git /opt/social-eagle-git
cd /opt/social-eagle-git
```

Create the server-side .env

```
cd /opt/social-eagle-git
nano .env
# Add the variable names required by the app, with your private values:
# OPENAI_API_KEY=... (enter locally; never share it)
# NEO4J_URI=...
# NEO4J_USERNAME=...
# NEO4J_PASSWORD=...
# NEO4J_DATABASE=...
# In nano: Ctrl+O, Enter to save; Ctrl+X to exit.
chmod 600 .env
```

Build and start the service

```
cd /opt/social-eagle-git
docker compose up --build -d
docker compose ps
```

Expected result
The social-eagle service should show Up. If it is restarting or exited, read docker compose logs --tail=80 social-eagle and check the server .env. Do not copy the .env into GitHub.

8. Connect the domain and HTTPS

Buy or register a domain, then open its DNS records in the registrar or DNS provider. Here the domain is managed in Hostinger and is named sedeploy.tech.

Create these DNS records

Type	Name	Value / target	Purpose
A	@	187.127.134.163	Routes sedeploy.tech to the VPS IPv4 address.
CNAME	www	sedeploy.tech	Routes www.sedeploy.tech to the root domain.

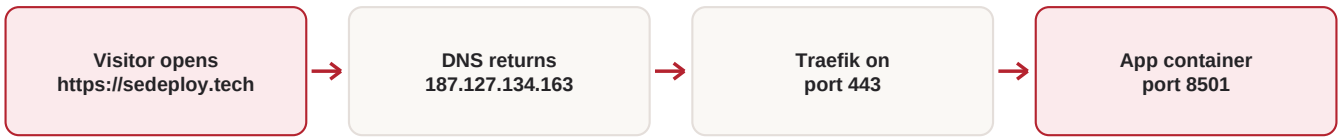
In Hostinger, open the domain DNS management page. Add or edit an A record for @ with the VPS IP. Add or edit a CNAME record for www pointing to sedeploy.tech. Remove conflicting duplicate A or CNAME records for these same names. Save, then allow time for DNS to update.

Verify DNS from Windows PowerShell or Command Prompt

```
nslookup sedeploy.tech
nslookup www.sedeploy.tech
```

Both names should resolve to 187.127.134.163. The VPS Traefik config listens on port 80 (web) and 443 (websecure), redirects HTTP to HTTPS, and uses the letsencrypt certificate resolver with an HTTP challenge. The Compose labels connect the app to that router.

Request route



Why Nginx did not start
Nginx reported that port 80 was already in use. Inspection showed Traefik owned that port. This was expected for the existing VPS setup: do not stop Traefik or try to run another proxy on port 80. The site is already working through Traefik.

One security setting to review
The current Compose file publishes port 8501 on the VPS, so direct-IP access may work without HTTPS. After the domain works, use the Hostinger firewall to block public inbound port 8501 while keeping required SSH, HTTP (80), and HTTPS (443) access. Then verify https://sedeploy.tech still works. Do not expose database ports publicly.

Live check

Open https://sedeploy.tech and confirm the browser shows a secure connection and the app answers a document question. The www name should also route to the app.

9. Change the app and update hosting

For normal code or interface changes, the domain, DNS, Traefik, and certificate do not need to be changed. Repeat this local-to-live loop.

The update loop

A. On the Windows computer

```
cd E:\SocialEagleDeploy
docker compose up --build -d
# Check http://localhost:8501 in a browser
git status --short
git add 01Frontend/app.py       # for interface changes
git add compose.yaml            # only if Compose changed
git commit -m "Describe the change"
git push
```

Stage only the file you changed. Do not stage .env. Check that the commit succeeds and git push reports the main branch updated.

B. On the Hostinger VPS browser terminal

```
cd /opt/social-eagle-git
git pull
docker compose up --build -d
docker compose ps
docker compose logs --tail=50 social-eagle
```

Then verify
Refresh https://sedeploy.tech. Confirm the new interface or feature is present and ask one known question. If GitHub prompts for authentication, use the GitHub username and repo-scoped PAT at the prompts.

If only a secret changes: update .env on the VPS directly using nano, save it, keep chmod 600, and recreate the service with docker compose up -d. Do not commit .env or send its contents to anyone.

10. Copy this prompt for your next rebuild

Paste this into a new chat when you want guided help. The assistant should wait for your result after each step, so you always know what happened before moving on.

```
I am a beginner rebuilding my Spotify Architecture Assistant project. Please guide me interactively, one small step at a time, using s

Project facts:
- Local project folder: E:\SocialEagleDeploy
- GitHub repository: https://github.com/nithyakalaiovian/SocialEagleDeploy, branch main
- VPS project folder: /opt/social-eagle-git
- Domain: sedeploy.tech; www.sedeploy.tech points to the root domain
- VPS IPv4 address: 187.127.134.163
- App: one Streamlit Docker service, port 8501
- PDF: spotify_web_app_architecture.pdf in the project root
- RAG files: 02Backend/ingest.py, rag.py, chunks.json, and faiss_index/
- Search: FAISS meaning search, BM25 word search, and Neo4j graph facts
- Existing VPS proxy: Traefik handles ports 80 and 443 and HTTPS certificates
- App secret settings are in local and VPS .env files, which must never be committed or pasted into chat

Help me check or rebuild in this order: verify the folder and tools; confirm .env exists without asking me to reveal values; prepare t
```

How to work with the assistant
Copy the whole prompt. Then describe where you are: starting from the PDF, testing locally, or updating the live site. Share error messages only after removing any passwords or tokens.

11. Troubleshooting and final checklist

Check one layer at a time: local app, GitHub source, VPS container, DNS, or Traefik.

Symptom	Check / action
Local site does not load	Run docker compose ps, then docker compose logs --tail=50 social-eagle. Confirm Docker Desktop is running.
Compose YAML error	From E:\SocialEagleDeploy run docker compose config -q. Check indentation and quote labels consistently.
VPS says GitHub authentication failed	Use the GitHub username and a valid repo-scoped PAT at the password prompt. Verify repository access and token expiry. Do not use your GitHub account password.
VPS app container restarts	Run cd /opt/social-eagle-git then docker compose logs --tail=80 social-eagle. Confirm the server .env exists and has the expected variable names.
Browser shows an old UI	Confirm local file was saved, git push succeeded, VPS git pull advanced, docker compose up --build -d completed, then hard-refresh the browser.
HTTPS/domain fails	Check both nslookup results, docker compose ps, and that Traefik is still running. Do not install another service on port 80/443.

Final go-live checklist

Check	Expected result
Local Docker	App and FAQ selector work at http://localhost:8501.
Git hygiene	.env is absent from git status and the repository.
VPS	social-eagle container is Up after rebuild.
DNS	sedeploy.tech and www.sedeploy.tech resolve to 187.127.134.163.
HTTPS	https://sedeploy.tech loads with a valid browser padlock and answers questions.

Deployment completed
The app is live at https://sedeploy.tech. Your normal future workflow is: edit, test locally, commit and push, pull on the VPS, rebuild, and verify.